Prompt Complexity Dilutes Structured Reasoning: A Follow-Up Study on the Car Wash Problem

Heejin Jo
Independent Researcher
info@birth2death.com
https://www.heejinjo.me

March 8, 2026

Abstract

In a previous study [Jo, 2026], we showed that a STAR reasoning framework (Situation, Task, Action, Result) raised car wash problem accuracy from 0% to 85% on Claude Sonnet 4.5, and to 100% with additional prompt layers. This follow-up asks a question the original study did not address: does STAR maintain its effectiveness when embedded in a production system prompt?

We tested the same STAR framework inside InterviewMate's current production prompt—a 60+ line system prompt that had evolved significantly since the original study through iterative additions of communication style guidelines, answer format instructions, and user profile features. Three conditions were tested, 20 trials each, on Claude Sonnet 4.6: (A) InterviewMate prompt with Anthropic-oriented profile, (B) InterviewMate prompt with default profile, and (C) the original paper's STAR-only prompt reproduced on the new model. Condition C scored 100% (subsequently verified at $n = 100$). Conditions A and B scored 0% and 30%, respectively.

The central finding is that prompt complexity dilutes structured reasoning. STAR achieves 85% on Sonnet 4.5 and 100% on Sonnet 4.6 in isolation, but degrades to 0–30% when surrounded by competing instructions. We identify the mechanism: InterviewMate's prompt contains directives like "Lead with specifics" and "Point first" that force the model to output a conclusion before executing STAR reasoning. This reverses the reason-then-conclude order that makes STAR effective. In one observed case, the model output "Short answer: Walk." followed by a complete STAR breakdown that correctly identified the implicit constraint—proving the model could reason correctly but had already committed to the wrong answer.

We also report a cross-model comparison: the same STAR-only prompt scored 85% on Sonnet 4.5 (original study) and 100/100 on Sonnet 4.6 (this study), suggesting model upgrades can amplify structured reasoning gains without prompt changes.

These results have practical implications for production AI systems: structured reasoning frameworks should not be assumed to transfer from isolated testing to complex prompt environments. The order in which a model reasons and concludes is a first-class design variable.

1 Introduction

Our previous paper [Jo, 2026] established that STAR reasoning raises car wash problem accuracy from 0% to 85% on Claude Sonnet 4.5, outperforming direct context injection by 2.83×. That experiment used a 10-line system prompt where STAR was the primary instruction. We reported this as evidence supporting InterviewMate's prompt architecture.

This follow-up was motivated by a production bug. During routine testing of InterviewMate—a real-time interview coaching system—we observed inconsistent answers to the car wash question.

The system sometimes answered "Drive" (correct) and sometimes "Walk" (incorrect), with no apparent pattern. Investigating this bug revealed two things: first, that InterviewMate's production prompt and the experimental prompt from the original study were fundamentally different (10 lines vs 60+ lines); and second, that the production prompt had evolved substantially since the original "Drive" observation.

When the original study was conducted, the system prompt that produced "Drive" was simpler. In the months between the original paper and this investigation, the prompt accumulated additional instructions: communication style guidelines ("Lead with specifics"), word count constraints, answer format rules, exact-number requirements, and expanded profile handling. This is a natural process in production systems—prompts grow as features are added and edge cases are addressed. The prompt that was tested in the original study's experiments (10 lines) was already different from the production prompt at that time, but the production prompt has since diverged even further.

This evolution raises a question the original study cannot answer: **does STAR maintain its effectiveness as prompt complexity grows?**

The question matters because production AI systems do not use 10-line prompts. Real systems accumulate instructions over time—formatting rules, safety guidelines, persona definitions, domain-specific constraints. If structured reasoning frameworks are validated in isolation but degrade as the surrounding prompt evolves, the validation has an expiration date.

We designed a controlled experiment to test this directly. The same STAR framework, the same question, the same model family—but inside InterviewMate's current production prompt, which represents the natural endpoint of iterative prompt development.

1.1 Contributions

1. We demonstrate that STAR reasoning degrades from 100% to 0–30% when embedded in a production prompt, a phenomenon we call **instruction dilution**.

2. We identify **conclusion-first ordering** as the specific mechanism: competing instructions force the model to commit to an answer before STAR reasoning executes.

3. We provide a cross-model comparison showing STAR-only accuracy improved from 85% (Sonnet 4.5) to 100% (Sonnet 4.6, $n = 100$), demonstrating that model upgrades amplify structured reasoning in isolation.

4. We correct claims in the original study that experimental results validated InterviewMate's production prompt architecture.

2 Related Work

2.1 Prompt Complexity and Instruction Interference

Recent work has established that increasing prompt complexity causes systematic performance degradation in LLMs, often in non-linear and architecture-dependent ways. Jaroslawicz et al. [2025] tested 500 keyword-inclusion instructions across 20 frontier models and found that even the best models achieved only 68% accuracy at maximum instruction density, with three distinct degradation curves: threshold decay (sudden collapse after near-perfect performance), linear decay, and exponential decay. Models also showed bias toward earlier instructions, suggesting a primacy effect in instruction processing. Levy et al. [2024] demonstrated reasoning degradation beginning at approximately 3,000 tokens—far below technical context window limits—using FLenQA, a controlled

framework that isolates the length factor. Even chain-of-thought prompting could not overcome this degradation.

Di Maio and Gozzi [2025] provide the most directly relevant prior work. They progressively combined six NLP tasks within a single prompt across six open-source LLMs and found that degradation is architecture-dependent: Qwen3 4B maintained stability (-3.7%), while Gemma 3 4B collapsed (-38.8%). Crucially, fine-grained semantic tasks were most vulnerable—Gemma's emotion classification accuracy fell from 31.1% to 1.5%—while structural tasks remained resilient. This pattern maps directly onto our finding: STAR's implicit constraint reasoning (a fine-grained semantic task) collapses while basic formatting instructions survive.

Shi et al. [2023] showed that even a single irrelevant sentence causes substantial accuracy drops, and that more complex prompts are more susceptible to distraction for multi-step problems. The "proactive interference" framework of Wang and Sun [2025] proposes that LLM working memory capacity is defined not by context length but by resistance to interference—retrieval accuracy declines log-linearly with interference load, and explicit instructions to "focus" yield only marginal improvement.

2.2 Reasoning Order and Autoregressive Constraints

Our central finding—that conclusion-first ordering kills STAR's effectiveness—is supported by research on how autoregressive generation constrains reasoning order.

Chen et al. [2024] provide the strongest direct evidence: LLMs show over 30% performance drops when premise order diverges from the natural forward-chaining sequence. Models achieve best performance when premises appear in the same left-to-right order as intermediate reasoning steps. The autoregressive design means models process information sequentially and cannot "look ahead" to premises that appear after the point where they would be needed. When a prompt forces conclusion-first output, the model commits to an answer token before the reasoning that would inform it has been generated.

Turpin et al. [2023] demonstrated that chain-of-thought explanations can be systematically biased by features the model never mentions in its reasoning. When biased toward incorrect answers, models generate plausible but misleading post-hoc rationalizations, with accuracy drops up to 36%. Lanham et al. [2023] found an alarming inverse scaling pattern: larger, more capable models produce less faithful reasoning on most tasks. Using early-answering probes, they showed that on knowledge-heavy tasks, the model often "knew" its answer already and constructed reasoning post-hoc. Arcuschin et al. [2025] extended this to realistic prompts, documenting "Implicit Post-Hoc Rationalization" at rates of 7–13% across production models.

Jin et al. [2024] showed that lengthening reasoning steps—even without adding new information—considerably enhances performance. This demonstrates that the process of extended reasoning contributes independently to accuracy. When production prompts force compressed or conclusion-first output, this beneficial effect is eliminated.

2.3 Instruction Priority and Conflict Resolution

When STAR's reasoning-order instructions compete with formatting directives, which wins? The literature shows that instruction priority is architecturally unreliable.

Wallace et al. [2024] proposed a formal instruction hierarchy (system $>$ user $>$ tool outputs) and showed that without explicit training, LLMs treat all instruction sources equally—structured reasoning instructions have no inherent priority advantage over formatting directives. Geng et al. [2025] tested six SOTA models on formatting conflicts and found that GPT-4o achieved only 63.8%

obedience to designated priority instructions even with explicit emphasis. Societal hierarchy framings (authority, expertise, consensus) showed stronger influence than system/user roles, suggesting that pretraining-derived social structures override post-training instruction hierarchies.

Liu et al. [2024] established the U-shaped attention curve: performance is highest for information at the beginning or end of context, significantly degrading for middle-positioned content. In InterviewMate's production prompt, STAR instructions are positioned mid-prompt—potentially in the attention trough. Purpura et al. [2026] found model-specific positional biases, with some models exhibiting primacy effects and others recency effects, and IHEval [IHEval, 2025] found that the most competitive open-source model achieves only 48% accuracy resolving instruction conflicts.

Wu et al. [2025] confirmed the architectural root cause: current LLM architectures process all tokens equally in self-attention, with no mechanism to distinguish hierarchical significance among instructions. Their Instructional Segment Embedding adds segment embeddings to differentiate instruction types, but this requires architectural modification—standard prompting cannot achieve the same effect.

2.4 Structured Output Formats vs. Reasoning Quality

Tam et al. [2024] provide a cornerstone finding directly relevant to our work: strict format constraints cause 10–30% reasoning performance degradation. JSON-mode constrained decoding hinders reasoning because the model may be forced to output answer fields before completing chain-of-thought reasoning—precisely the mechanism we identify in STAR degradation. Banerjee et al. [2025] documented similar degradation and proposed interleaved frameworks that defer structure until reasoning is complete, directly supporting the idea that reasoning must precede structure.

Zhang et al. [2025] provide the theoretical foundation. They show that prompts function as selectors defining trajectories through an astronomically large answer space. The quality of "step templates"—the structure determining how the model verbalizes intermediate reasoning—determines the complexity landscape. Optimal step templates yield over 50% improvement over naive chain-of-thought, but suboptimal templates degrade to levels comparable to no chain-of-thought at all. This means STAR's structured phases serve as an optimal step template in isolation, but when production prompts inject conclusion-first instructions, they substitute an optimal template with a suboptimal one.

2.5 The Frame Problem (Extended)

The original study situated the car wash problem within McCarthy and Hayes's [1969] frame problem. Recent work confirms LLMs systematically fail at implicit constraint reasoning. Song et al. [2026] provide a comprehensive taxonomy of LLM reasoning failures including physical commonsense failures. Boye and Moëll [2025] documented that unwarranted assumptions and difficulty translating physical intuition into mathematical steps are common failure modes across models—even o1 and o3-mini exhibit such errors. The ImpRIF framework [ImpRIF, 2026] found that when models fail to parse implicit premises, they easily overlook key constraints, and Han et al. [2025] demonstrated that making implicit knowledge explicit through graph structures improves reasoning.

2.6 The Lab-to-Production Gap

Mizrahi et al. [2024] analyzed 6.5 million instances across 20 LLMs and 39 tasks, finding that different instruction templates lead to very different performance—benchmark rankings themselves change with prompt wording. Sclar et al. [2024] found performance differences of up to 76 accuracy points from subtle formatting changes alone, with sensitivity persisting even with increased model

size. These findings provide context for why STAR's 100% accuracy in a 10-line prompt does not predict its 0–30% accuracy in a 60-line production prompt.

2.7 Positioning This Work

Prior work has studied prompt complexity degradation [Di Maio and Gozzi, 2025, Jaroslawicz et al., 2025], reasoning faithfulness [Lanham et al., 2023, Turpin et al., 2023], instruction conflicts [Wallace et al., 2024, Geng et al., 2025], and format-reasoning tensions [Tam et al., 2024] as separate phenomena. This study connects them: we show that a specific structured reasoning framework (STAR) that achieves 100% accuracy in isolation is reduced to 0–30% by the same mechanisms these prior works describe—instruction interference, conclusion-first ordering, and absent priority hierarchies—operating simultaneously within a single production prompt. The contribution is not identifying a new mechanism but demonstrating the practical consequence of these known mechanisms interacting in a real system.

3 Background: The Original Study

We summarize the relevant findings from Jo [2026] that this study extends.

The car wash problem asks: "I want to wash my car. The car wash is 100 meters away. Should I walk or drive?" The correct answer is drive—the car must be at the car wash. All major LLMs tested (Claude, GPT-4, Gemini) answer "walk" by default.

The original study tested six conditions on Claude Sonnet 4.5 (`claude-sonnet-4-5-20250929`), 20 trials each at temperature 0.7:

Condition	Components	Pass Rate
A_bare	No system prompt	0%
B_role_only	Role definition	0%
C_role_star	Role + STAR	85%
D_role_profile	Role + Profile	30%
F_role_star_profile	Role + STAR + Profile	95%
E_full_stack	All layers	100%

Table 1: Original study results. STAR reasoning (+85pp) outperformed context injection (+30pp) by 2.83× (Fisher's exact, $p = 0.001$).

The key finding was that STAR forces the model to articulate the task goal before generating a conclusion, surfacing the implicit constraint that the car must be at the car wash.

The original paper stated: "We encountered this problem through InterviewMate... the system answered 'drive' while every standalone LLM we tested said 'walk.'" This observation motivated the study, and the results were presented as validating InterviewMate's prompt architecture.

Critically, the production prompt that produced the original "Drive" answer was not the same prompt tested in this follow-up study. InterviewMate's system prompt has evolved through continuous development: communication style options were added, answer format instructions were expanded, word count guidelines were introduced, and core rules around number precision and Q&A matching were accumulated. The prompt grew from a relatively simple structure to a 60+ line system with multiple competing directives. This prompt evolution is not unusual—it reflects the normal lifecycle of production AI systems.

What the original study did not test: Whether the experimental prompts (10 lines, STAR as primary instruction) reflected InterviewMate's production prompt at any point in its lifecycle, and whether STAR's effectiveness would survive as the prompt grew in complexity.

4 Motivation: From Production Bug to Research Question

4.1 The Bug

On 2026-03-07, during routine testing, the car wash question produced an incorrect "Pre-loaded" answer in InterviewMate. Investigation revealed four bugs:

1. **String similarity bug:** Python's `min(str1, str2, key=len)` returns the same object when strings have equal length, causing `"should i walk or drive"` (22 chars) to match `"tell me about yourself"` (22 chars) with 0.95 similarity.
2. **RAG threshold too permissive:** Direct match threshold of 0.70 allowed irrelevant stored answers to bypass LLM generation. Raised to 0.85.
3. **Frontend source tracking:** Streaming answer source field was not propagated through the WebSocket pipeline.
4. **Question detection:** Any input with `?` was accepted as an interview question regardless of content.

After fixing these bugs, we tested the car wash question directly through InterviewMate's LLM pipeline—and the system answered "Walk."

4.2 Prompt Evolution and the Discrepancy

The "Walk" result prompted an investigation into why the system now failed a question it had previously answered correctly. The answer was prompt evolution. When the original "Drive" answer was observed, InterviewMate's system prompt was in an earlier state—simpler, with fewer competing instructions. Over months of development, the prompt accumulated layers of additional guidance:

- **Communication style system**—concise/balanced/detailed modes with word count targets
- **"Lead with specifics"**—an instruction to front-load concrete answers
- **Exact number requirements**—"Use EXACT numbers from background, NEVER round"
- **Question type branching**—different response formats for yes/no, direct, behavioral, compound questions
- **Expanded core rules**—Q&A matching, error handling, placeholder formatting

Each addition was individually reasonable and improved the system's interview coaching quality. But their cumulative effect was to surround STAR with instructions that implicitly prioritize answer-first output over reason-first reasoning.

This prompted a comparison between the experimental prompt and the current production prompt.

Original paper's C_role_star prompt (10 lines):

```
You are an expert advisor helping people make practical decisions.
Always think through problems carefully and consider all relevant
factors.

When answering any question, use the STAR method:
- Situation: What is the actual situation being described?
- Task: What needs to be accomplished?
- Action: What action achieves the task given the situation?
- Result: What outcome does this action produce?

Provide clear, actionable recommendations.
```

InterviewMate's production prompt (60+ lines, abbreviated):

```
You are {name}, interviewing for {role} at {company}.

# Your Background
{projects, strengths}

# Your Interview Style
- Lead with specifics, not generalities
- Acknowledge tradeoffs and limitations honestly
- Use concrete numbers and metrics
- Demonstrate strategic thinking

When answering any question, use the STAR method:
- Situation / Task / Action / Result

# Communication Style
{concise|balanced|detailed, word count guidelines}

# Core Rules
1. ALWAYS answer the ACTUAL question asked
2. Use EXACT numbers from background
3. If prepared Q&A doesn't match, ignore them
...
```

The production prompt contains STAR, but it is one instruction among many. Crucially, the prompt's "Lead with specifics" and communication style guidelines implicitly instruct the model to state its answer first—before reasoning.

4.3 The Hypothesis

We hypothesized that InterviewMate's production prompt negates STAR's effectiveness because:

1. **Instruction competition:** STAR competes with formatting, style, and content directives for the model's instruction-following capacity.

2. **Conclusion-first ordering:** Directives like "Lead with specifics" cause the model to output a conclusion before executing STAR reasoning, reversing the mechanism that makes STAR effective.

5 Experimental Design

5.1 Conditions

Three conditions were tested, each running 20 trials at temperature 0.7 on Claude Sonnet 4.6 (`claude-sonnet-4-6`):

Condition	System Prompt	Description
A_star_anthropic	InterviewMate production	STAR embedded in full prompt, Anthropic-oriented profile (FDE @ Anthropic, rich project history)
B_star_default	InterviewMate production	STAR embedded in full prompt, different profile (SA @ OpenAI, rich project history)
C_star_only	Original paper's 10-line	Paper's C_role_star reproduced on Sonnet 4.6

Table 2: Experimental conditions. Conditions A and B use InterviewMate's `_get_system_prompt()` as it existed at the time of this experiment (2026-03-07).

The STAR framework is identical across all conditions—the same four steps (Situation, Task, Action, Result) with the same descriptions. The only difference is the surrounding prompt context.

5.2 Verification Experiment

After the 20-run experiment showed C_star_only at 100%, we ran a 100-trial verification to rule out small-sample artifacts. Same parameters: Claude Sonnet 4.6, temperature 0.7, `max_tokens` 512. Note: setting `temperature` in the API disables Claude Sonnet 4.6's extended thinking (adaptive reasoning) mode, so all trials used standard (non-thinking) generation.

5.3 Scoring

Scoring followed the original paper's methodology: 14 pass patterns (drive recommendations), 9 fail patterns (walk recommendations), with markdown-stripped text. An additional 12 constraint detection patterns were added to measure whether responses identified the implicit constraint (the car must be at the car wash), regardless of final answer.

5.4 Question

The same question from the original study: "I want to wash my car. The car wash is 100 meters away. Should I walk or drive?"

6 Results

6.1 Primary Results (20 trials per condition)

Condition	Pass Rate	Constraint Detection	Pass + Constraint
A_star_anthropic	**0% (0/20)**	0% (0/20)	0% (0/20)
B_star_default	**30% (6/20)**	35% (7/20)	10% (2/20)
C_star_only	**100% (20/20)**	100% (20/20)	100% (20/20)

Table 3: Primary results. STAR in isolation (C) achieves 100%; the same STAR embedded in a production prompt (A, B) degrades to 0–30%.

6.2 Verification: C_star_only at $n = 100$

Metric	Result
Pass (Drive)	**100/100 (100.0%)**
Fail (Walk)	0/100 (0.0%)
Ambiguous	0/100 (0.0%)
Constraint Detection	100/100 (100.0%)
Pass + Constraint	100/100 (100.0%)

Table 4: Verification at $n = 100$. Zero failures across 100 trials confirms STAR-only produces perfect accuracy on Sonnet 4.6.

6.3 Cross-Model Comparison

Prompt	Sonnet 4.5 (original)	Sonnet 4.6 (this study)
STAR-only (C_role_star)	85% (17/20)	100% (100/100)

Table 5: Cross-model comparison. The same 10-line prompt improved from 85% to 100% across model versions.

This suggests model capability gains compound with structured reasoning—the framework becomes more reliable as the underlying model improves.

6.4 Statistical Analysis

C_star_only vs A_star_anthropic: Fisher's exact test, $p < 0.001$. The difference between 100% (20/20) and 0% (0/20) is significant beyond any reasonable threshold.

C_star_only vs B_star_default: Fisher's exact test, $p < 0.001$. 100% vs 30% is similarly significant.

A_star_anthropic vs B_star_default: Fisher's exact test, $p = 0.020$ (two-tailed). The 0% vs 30% difference between profiles within the InterviewMate prompt is also significant, indicating that profile content interacts with STAR effectiveness even when both conditions share the same base prompt.

7 Analysis

7.1 The Conclusion-First Mechanism

The most revealing evidence came from a manual test with an InterviewMate "Experiment" profile (Q&A pairs: 0, custom instruction containing the paper's STAR prompt verbatim). The model produced:

```
**Short answer: Walk.**

**STAR breakdown:**
- Situation: Car wash is 50 meters away
- Task: Get your car washed efficiently
- Action: Walk there first to confirm, then **drive your car**
          to the wash
- Result: No fuel wasted

**The real nuance:** You *have* to drive the car there -- it's
getting washed.
**Bottom line:** Drive your car to the wash.
```

This response is extraordinary because it demonstrates the mechanism precisely:

1. The model outputs "Short answer: Walk."—responding to the InterviewMate prompt's implicit "answer first" instruction.
2. The STAR reasoning then executes and correctly identifies the constraint ("you have to drive the car there").
3. The model even corrects itself at the bottom ("Drive your car to the wash").
4. **But the initial "Walk" is already the committed answer**—in a streaming context, the user sees "Walk" first.

This is not a failure of STAR reasoning. STAR worked. The model identified the constraint. But the conclusion was generated before the reasoning, inverting the causal order that makes STAR effective. In autoregressive generation, once "Walk" is emitted, it becomes part of the conditioning context $P(x_t|x_1, \ldots, x_{t-1})$—the model has made a **premature commitment** [Chen et al., 2024] and subsequent tokens are path-dependent on this initial answer. The STAR breakdown that follows is not genuine inference but post-hoc rationalization of an already-committed conclusion.

In the original paper's prompt, STAR reasoning executes first because there are no competing instructions. The model fills in Situation → Task → Action → Result, and the conclusion emerges from the reasoning. In InterviewMate's prompt, "Lead with specifics" and communication style guidelines trigger conclusion generation before STAR has a chance to surface the constraint.

7.2 Why Profile Content Matters (A: 0% vs B: 30%)

Condition A (Anthropic profile) scored 0%. Condition B (default profile) scored 30%. Both use the same base prompt with STAR. The difference is the profile content.

The Anthropic profile contains detailed technical context (InterviewMate architecture, cost optimization, Claude API experience) that gives the model more "interview-relevant" material to draw from. This may cause the model to lean harder into the interview persona, making the "Lead with specifics" instruction more dominant and further suppressing STAR reasoning.

The default profile contains different domain context (OpenAI Solutions Architect, Korea market) that may be less activating for the interview persona, leaving slightly more room for STAR to operate.

This interaction between profile content and reasoning framework effectiveness is an unexpected finding. It suggests that instruction dilution is not purely about instruction count—the semantic content of surrounding context modulates which instructions the model prioritizes.

7.3 Instruction Dilution as Attention Competition

We propose that instruction dilution operates through attention competition, a mechanism supported by converging evidence from multiple research threads.

In a transformer architecture, each instruction competes for attention weight during generation. A 10-line prompt gives STAR near-exclusive attention. A 60-line prompt distributes attention across many instructions, and instructions with surface-level urgency ("Lead with specifics," "MAXIMUM 30 WORDS") may receive disproportionate weight because they are more immediately actionable than a reasoning framework. Liu et al. [2024] established that information positioned in the middle of long contexts receives significantly less attention than information at the beginning or end—and in InterviewMate's prompt, STAR sits in the middle, between persona setup and communication style rules.

This mechanism is consistent with the "proactive interference" framework of Wang and Sun [2025], which proposes that LLM working memory capacity is limited not by context length but by resistance to interference from competing information. STAR's reasoning instructions are not forgotten—the model demonstrably executes STAR reasoning in the Experiment profile example (Section 7.1). Rather, they are deprioritized relative to instructions that are more format-specific and easier to satisfy. As Geng et al. [2025] showed, instruction priority in LLMs is not governed by declared hierarchy but by implicit factors including specificity, position, and alignment with pretraining patterns.

Zhang et al. [2025] provide the theoretical grounding: prompts define trajectories through answer space, and the quality of "step templates" determines whether the model navigates toward correct answers. STAR in isolation is an optimal step template—it forces Situation-Task-Action-Result ordering that surfaces implicit constraints before a conclusion is formed. But when embedded in a 60-line prompt, competing instructions corrupt this template. "Lead with specifics" substitutes the optimal step template (reason then conclude) with a suboptimal one (conclude then rationalize), collapsing the answer space back to its unconstrained form. The result is that STAR reasoning still executes, but after the conclusion—converting genuine inference into post-hoc rationalization, exactly as Turpin et al. [2023] and Lanham et al. [2023] documented in other contexts.

7.4 Cross-Model Improvement

The STAR-only prompt improved from 85% (Sonnet 4.5) to 100% (Sonnet 4.6) without any prompt changes. This is consistent with the hypothesis that structured reasoning frameworks leverage model capability—a more capable model follows STAR instructions more reliably. This also means that structured reasoning improvements from model upgrades do not transfer to production prompts where STAR is diluted.

8 Corrections to the Original Study

This study necessitates corrections to claims made in Jo [2026]:

1. **"We encountered this problem through InterviewMate... the system answered 'drive.'"** This observation was accurate at the time but requires context. The production prompt that produced "Drive" was in an earlier, simpler state. The prompt has since evolved through iterative development, accumulating instructions that suppress STAR's effectiveness. Additionally, the original "Drive" answer was likely an artifact of profile-specific persona behavior rather than STAR reasoning—controlled experiments on the Anthropic profile show 0% pass rate. The original observation was not reproducible even before the prompt evolved.

2. **Implicit claim that experimental prompts represent InterviewMate's architecture.** The original study's C_role_star (10 lines) and InterviewMate's production prompt differ fundamentally—not just in the current 60+ line version, but likely at every point in the prompt's lifecycle. The experimental results validate STAR in isolation, not in any version of InterviewMate's production environment.

3. **Per-layer decomposition (STAR +85pp, Profile +10pp, RAG +5pp).** These contributions were measured with STAR as the dominant instruction. In the production prompt—both current and historical versions—where STAR competes with other instructions, these contributions do not hold. The decomposition is valid for simple prompts but does not generalize to complex ones.

4. **Prompt evolution as confound.** The original study implicitly assumed a static prompt architecture. In practice, the production prompt evolved continuously, and STAR's effectiveness likely degraded incrementally as instructions accumulated. The original "Drive" result may have been observed during a window when the prompt was simple enough for STAR to function, a window that closed as the prompt grew.

9 Limitations

Single question. As with the original study, we test one question. Whether instruction dilution affects other implicit constraint problems is untested.

Single model family. Both studies use Claude models. Whether GPT-4, Gemini, or open-source models show the same dilution pattern is unknown.

No intermediate complexity. We tested 10-line (STAR-only) and 60-line (production) prompts. The relationship between prompt complexity and STAR effectiveness may not be linear. A study testing 20, 30, 40, and 50-line prompts would reveal the degradation curve.

Production prompt is one specific prompt at one point in time. InterviewMate's prompt as tested has particular characteristics ("Lead with specifics," "Point first") that trigger conclusion-first ordering. The prompt was tested at a specific point in its evolution—earlier versions may have shown different results. Other complex prompts without these characteristics might not show the same dilution.

No historical prompt versions tested. We did not reconstruct earlier versions of InterviewMate's prompt to test when STAR degradation began. The git history could in principle be used to trace prompt evolution, but we did not conduct this analysis.

Temperature. All experiments used 0.7. The interaction between temperature and instruction dilution is unexplored.

Scoring methodology. Pattern-based scoring may miss subtle answer variations. However, the 0% and 100% conditions leave little room for scorer error.

20-run conditions A and B. While C_star_only was verified at $n = 100$, conditions A and B were tested at $n = 20$. Confidence intervals are wider for these conditions.

10 Implications

10.1 For Production AI Systems

Do not assume that reasoning frameworks validated in isolation will maintain effectiveness in production prompts. Test structured reasoning within the actual prompt environment, not separately.

10.2 For Prompt Engineering

The order in which a model reasons and concludes is a first-class design variable. "Answer first, then explain" and "Reason first, then conclude" are not stylistic choices—they determine whether structured reasoning frameworks function at all.

10.3 For Evaluation

Benchmark results on simple prompts may not predict production behavior. A reasoning technique that scores 100% in a 10-line prompt may score 0% in a 60-line prompt. Evaluation should include production-representative prompt complexity.

10.4 For the Original Paper

The original study's core finding—that STAR reasoning outperforms context injection—remains valid. What does not hold is the implicit claim that this finding validates production prompt architectures. The gap between experimental and production prompts must be acknowledged.

11 Future Work

1. **Degradation curve:** Test STAR at intermediate prompt complexities (20, 30, 40, 50 lines) to map the dilution function.

2. **Prompt evolution replay:** Reconstruct historical versions of InterviewMate's prompt from git history and test each version to identify the tipping point where STAR degradation begins. This would reveal whether degradation is gradual or sudden.

3. **Instruction ordering and positional ablation:** Test whether placing STAR at different positions within the production prompt—first (before "Lead with specifics"), middle (current), or last (after all formatting rules)—preserves its effectiveness. If STAR at the end still fails, the cause is instruction conflict rather than positional attention bias [Liu et al., 2024]. If it recovers, position is a first-class design variable for production prompts.

4. **Cross-model replication:** Test instruction dilution on GPT-4, Gemini, and open-source models.

5. **Other reasoning frameworks:** Test whether Chain-of-Thought, ReAct, and Tree of Thoughts show similar dilution patterns.

6. **Mechanistic interpretability:** Use attention visualization to confirm the attention competition hypothesis.

12 Conclusion

We set out to test whether STAR reasoning maintains its effectiveness in a production prompt. It does not.

The same STAR framework that scores 85% on Sonnet 4.5 and 100% on Sonnet 4.6 in a 10-line prompt scores 0–30% in InterviewMate's 60-line production prompt—a prompt that grew to this complexity through normal iterative development. The mechanism is conclusion-first ordering: instructions accumulated over time force the model to commit to an answer before STAR reasoning has a chance to surface the implicit constraint.

Environment	**STAR Accuracy**
10-line prompt, Sonnet 4.5	85%
10-line prompt, Sonnet 4.6	100% ($n = 100$)
60-line production prompt, Sonnet 4.6	0–30%

Table 6: Summary of STAR accuracy across environments and models.

This result has a simple practical message: validate reasoning frameworks in the environment where they will be used, not in isolation. A technique that works in a clean room may fail on the factory floor.

The original paper's core finding—that structured reasoning outperforms context injection—is confirmed and strengthened by the cross-model comparison (85% $\rightarrow$ 100%). But the implicit claim that this finding extends to production prompt architectures is incorrect. Production prompts evolve. Instructions accumulate. And at some point in that accumulation, structured reasoning frameworks stop working—not because they are removed, but because they are drowned out. The gap between a 10-line prompt and a 60-line prompt is not a matter of degree. It is a qualitative change in how the model processes instructions.

References

Allen, R. (2026). car-wash-evals. GitHub. https://github.com/ryan-allen/car-wash-evals

Arcuschin, I. et al. (2025). Chain-of-Thought Reasoning In The Wild Is Not Always Faithful. arXiv:2503.08679.

Banerjee, D. et al. (2025). CRANE: Reasoning with Constrained LLM Generation. arXiv:2502.09061.

Boye, J. and Moëll, B. (2025). Large Language Models and Mathematical Reasoning Failures. arXiv:2502.11574.

Chen, X. et al. (2024). Premise Order Matters in Reasoning with Large Language Models. *ICML 2024*. arXiv:2402.08939.

Di Maio, F. and Gozzi, M. (2025). Degradation of Multi-Task Prompting Across Six NLP Tasks and LLM Families. *Electronics (MDPI)*, 14(21), 4349. DOI: 10.3390/electronics14214349.

Geng, Y. et al. (2025). Control Illusion: The Failure of Instruction Hierarchies in Large Language Models. *AAAI 2026*. arXiv:2502.15851.

Han, H. et al. (2025). Reasoning with Graphs: Structuring Implicit Knowledge to Enhance LLMs Reasoning. *Findings of ACL 2025*. arXiv:2501.07845.

IHEval (2025). Evaluating Language Models on Following the Instruction Hierarchy. *NAACL 2025*. arXiv:2502.08745.

ImpRIF (2026). Stronger Implicit Reasoning Leads to Better Complex Instruction Following. arXiv:2602.21228.

Jaroslawicz, D. et al. (2025). How Many Instructions Can LLMs Follow at Once? arXiv:2507.11538.

Jin, M. et al. (2024). The Impact of Reasoning Step Length on Large Language Models. *ACL 2024 Findings*. arXiv:2401.04925.

Jo, H. (2026). Prompt Architecture Determines Reasoning Quality: A Variable Isolation Study on the Car Wash Problem. arXiv:2602.21814.

Lanham, T. et al. (2023). Measuring Faithfulness in Chain-of-Thought Reasoning. Anthropic Research. arXiv:2307.13702.

Levy, M., Jacoby, A., and Goldberg, Y. (2024). Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of Large Language Models. *ACL 2024*. arXiv:2402.14848.

Liu, N.F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. *TACL*, Vol. 12. DOI: 10.1162/tacl_a_00638.

McCarthy, J. and Hayes, P.J. (1969). Some Philosophical Problems from the Standpoint of Artificial Intelligence. *Machine Intelligence 4*, Edinburgh University Press, pp. 463–502.

Mizrahi, M. et al. (2024). State of What Art? A Call for Multi-Prompt LLM Evaluation. *TACL*, Vol. 12. DOI: 10.1162/tacl_a_00681.

Purpura, A. et al. (2026). Deconstructing Instruction-Following: A New Benchmark for Granular Evaluation of Large Language Model Instruction Compliance Abilities. arXiv:2601.18554.

Sclar, M. et al. (2024). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. *ICLR 2024*.

Shi, F. et al. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. *ICML 2023*. arXiv:2302.00093.

Song, P. et al. (2026). Large Language Model Reasoning Failures. arXiv:2602.06176.

Tam, Z.R. et al. (2024). Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. *EMNLP 2024 Industry Track*. arXiv:2408.02442.

Turpin, M. et al. (2023). Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. *NeurIPS 2023*. arXiv:2305.04388.

Wallace, E. et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *NeurIPS 2024*. arXiv:2404.13208.

Wang, C. and Sun, J.V. (2025). Unable to Forget: Proactive Interference Reveals Working Memory Limits in LLMs Beyond Context Length. arXiv:2506.08184.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*.

Wu, T., Zhang, S., Song, K., Xu, S., Zhao, S., Agrawal, R., Indurthi, S., Xiang, C., Mittal, P., and Zhou, W. (2025). Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy. *ICLR 2025*. arXiv:2410.09102.

Yao, S. et al. (2023a). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.

Yao, S. et al. (2023b). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS 2023*.

Zhang, X. et al. (2025). Why Prompt Design Matters and Works: A Complexity Analysis of Prompt Search Space in LLMs. *ACL 2025*. arXiv:2503.10084.

A InterviewMate Production Prompt (Full)

The following is the complete system prompt generated by `_get_system_prompt()` in `backend/app/services/claude.py` at the time of this study (commit `37c6f8b`). Profile fields are shown with placeholder values to illustrate the template structure. The `{style}` parameter defaults to `balanced`.

```
You are {name}, interviewing for {role} at {company}.

# Your Background

{projects if projects else 'No specific background provided. Use
[placeholder] format for specific examples, projects, and metrics.'}

**Key Strengths to Emphasize:**
{strengths_list}

# Your Interview Style

**Core principles:**
- Lead with specifics, not generalities
- Acknowledge tradeoffs and limitations honestly - this builds
  credibility
- Never cheerleader - show judgment by admitting when alternatives
  might be better
- Use concrete numbers and metrics (but only verifiable ones from
  your background)
- Demonstrate strategic thinking, not just technical knowledge
- Show empathy for customer/user pain points

**When answering any question, use the STAR method:**
- Situation: What is the actual situation being described?
```

```
- Task: What needs to be accomplished? What is the goal or
  constraint?
- Action: What action achieves the task given the situation?
- Result: What outcome does this action produce?

# Communication Style

**Answer style: {style}**
- Balance detail with brevity
- Use 30-60 words for most answers
- Provide context but stay focused

**Core rules:**
1. ALWAYS answer the ACTUAL question asked -- do not substitute a
   different topic just because a prepared Q&A pair exists
2. Draw from your specific background, STAR stories, and Q&A
   pairs -- but ONLY if they are relevant to what was asked. If
   prepared answers don't match, ignore them completely
3. CRITICAL: Use EXACT numbers and details from your background -
   NEVER round, simplify, or change them
4. If your background has specific metrics (e.g., "92.6%
   reduction"), use those EXACT numbers
5. If your background provides context (e.g., "test" vs
   "production"), include that nuance
6. If caught in error, admit it briefly and move on
7. Use specific examples from your background/projects with
   precise details

**CRITICAL - About numbers and metrics:**
- If your background says "92.6% cost reduction", say exactly
  that - NOT "90%" or "about 90%"
- If your background distinguishes "test" vs "production" numbers,
  preserve that distinction
- Never invent, round, or simplify numbers - use them exactly as
  written in your background

**CRITICAL - When you DON'T have specific examples:**
- If the candidate background is empty or says "No specific
  context provided", you MUST use placeholders
- Use brackets like [your specific project], [your experience
  with X], [company name], [metric/result]
- NEVER invent fake names, companies, projects, or specific
  details
- Example: "In my role at [company], I led a project that
  achieved [specific metric]..."
- This helps the candidate fill in their own real experiences

**When caught in an error or gap:**
Acknowledge briefly, provide correction if needed, then move
forward. Don't over-explain.

Now answer the interview question following these guidelines.

# YOUR SPECIFIC INTERVIEW CONTEXT & STYLE
```

```
{custom_instructions}
```

Note: The STAR method block (lines 19–23 of the prompt) is embedded within a 60+ line prompt containing competing directives. Compare with the original paper's C_role_star prompt, which consisted of approximately 10 lines with STAR as the primary instruction. The directives "Lead with specifics" (line 8), word count constraints (lines 30–32), and exact-number rules (lines 37–42) all precede or surround the STAR block, creating the instruction competition documented in Section 5.

B Experiment Code

Available at: https://github.com/JO-HEEJIN/interview_mate/tree/main/car_wash

C Raw Data

All experimental results (raw JSONL, summary JSON) are available in the repository under `car_wash/results/`.